SLIIT
Discover Your Future

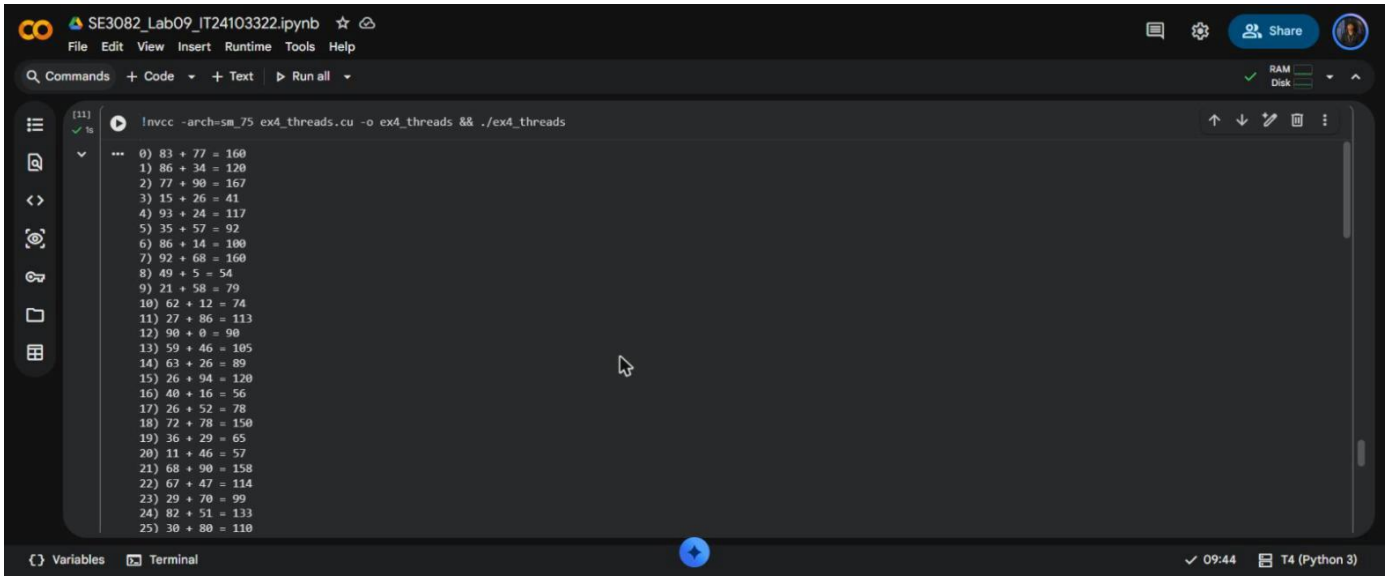
BSc (Hons) in Computer Science
Year 3

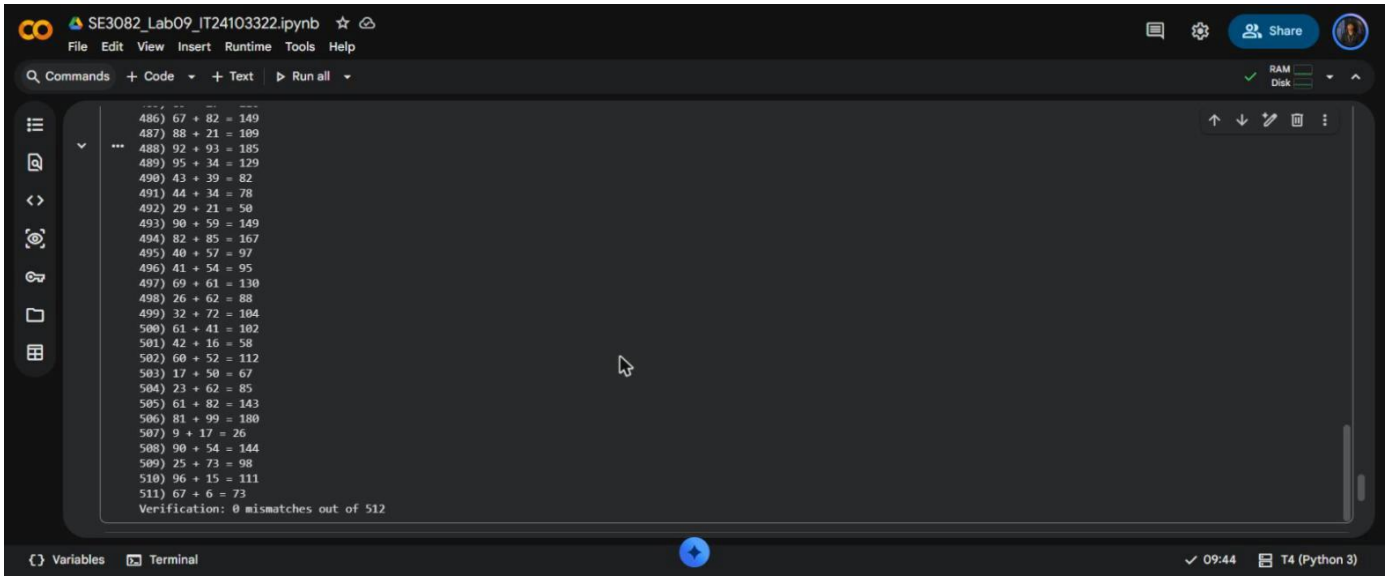
Lab Sheets

SE3082 – Parallel Computing **Semster 1, 2026**

IT24103322

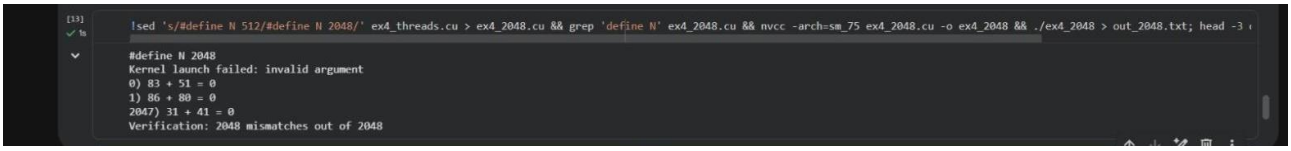
Exercise 4





The same addition with one block of 512 threads, addT<<<1, N>>>. Each thread uses threadIdx.x to choose its element. The first line is 0) 83 + 77 = 160, the last result is 511) 67 + 6 = 73 and the final line is Verification: 0 mismatches out of 512, as expected. The values are the same as Exercise 3 because the same rand() sequence is used.

1. **Change #define N 512 to #define N 2048, compile and run again. What is printed, which part of the program told you, and which limit of the GPU did you hit?**



To keep ex4_threads.cu unchanged, the cell copies it to ex4_2048.cu with N changed to 2048 (sed), then compiles and runs it, printing the first and last lines of the output. The program printed Kernel launch failed: invalid argument. This line comes from the cudaGetLastError() check after the launch; without it the program would carry on silently. Because the kernel never ran, every c value is 0 (for example 0) 83 + 51 = 0) and the result is Verification: 2048 mismatches out of 2048.

The limit is the maximum number of threads in one block, which is 1024 on the T4 (and on every current NVIDIA GPU). 2048 threads in one block is over that limit, so the launch configuration is rejected. On this CUDA 13.0 runtime the message is “invalid argument” rather than the “invalid configuration argument” wording in the sheet. N was left at 512 in ex4_threads.cu.

2. **One block of threads cannot cover a very large array.**

Because a block can hold at most 1024 threads, one block cannot cover an array larger than that. A large array needs many blocks of many threads together, with each thread working out its global position from blockIdx.x, blockDim.x and threadIdx.x. This is what Exercise 5 does.